Dealing with Data Corruption in PostgreSQL

Data corruption in PostgreSQL is a very unlilely, but possible scenario. Common causes of data corruption involves:

- Faulty Disks
- Faulty RAID controlers
- Defective RAM
- Usage of fsync=off and a OS crash or power loss
- OS bugs
- PostgreSQL bugs
- Administrator error

Detecting corrupted data

1. If data checksum is `off`, the most common way a corruption event will be detected is by looking for strings like this in the postgreSQL logs (after a query error):

```
could not read block ZZZZ in file "base/16401/33909": read only 0 of NNNN bytes
```

2. When data checksum is `on`, a different meesage could be seen in the logs:

```
WARNING: page verification failed, calculated checksum 726 but expected 827
ERROR: invalid page in block 0 of relation base/16401/33909
```

Important numbers to identify:

- 16401 represent the oid of the database, as in

```
gitlabhq_production=# select oid,datname from pg_database;
  oid  |       datname
-------+---------------------
 13052 | template0
 16400 | template1
 20485 | postgres
 16401 | gitlabhq_production
(4 rows)
```

- 33909 corresponds to what is called the `filenode`. You can find wich relation corresponds to `base/16401/33909` with the following query:

```
select n.nspname AS schema, c.relkind, c.relname AS relation from pg_class c inner join pg_n
```

```
 schema | relkind | relation
--------+---------+----------
 public | r       | users
(1 row)
```

Note: Common values for `relkind` column are:

- r for regular tables

- i for indexes
- t for TOAST tables

For more details about this, check the Official Documentation

What to when finding corrupted data

- If this has happen in a replica (and every other host is okay), the easiest way to solve this is by draining the traffic and recreate a new node, as in replica maintenance and recreate a replica
- If this problems affects the leader, then:
  - If the relation is an index, you could:
    - REINDEX CONCURRENTLY (only for PostgreSQL >=12), or
    - Re-create the index. (i)
  - If the relation is a table, you could:
    - verify wich row/s are affected (ii)
    - Contact DBRE and OnGres for support

(i) To recreate the index, you could follow this steps:

1. Find how this index is defined:

```
gitlabhq_production=# select pg_get_indexdef('index_on_users_lower_email'::regclass);
                                        pg_get_indexdef
-------------------------------------------------------------------------------------------
 CREATE INDEX index_on_users_lower_email ON public.users USING btree (lower((email)::text))
(1 row)
```

1. Use a different name for the new index, and use CONCURRENTLY:

```
CREATE INDEX CONCURRENTLY index_on_users_lower_email_new ON public.users USING btree (lower
```

1. Drop the corrupted index:

```
DROP INDEX CONCURRENTLY index_on_users_lower_email
```

1. Rename the new index to the droped one:

```
ALTER INDEX index_on_users_lower_email_new RENAME TO index_on_users_lower_email
```

(ii) To verify wich rows were affected by the corruption event: Probably the easiest way to do it is by using `FETCH_COUNT 1` in a `gitlab-psql session`. That allows results to be seen as it happens:

```
gitlabhq_production=# \set FETCH_COUNT 1
gitlabhq_production=# \pset pager off
Pager usage is off.
gitlabhq_production=# select ctid,* from users;
```

And check for errors, like

```
 (439,226) |           99878
 (439,227) |           99879
server closed the connection unexpectedly
        This probably means the server terminated abnormally
        before or while processing the request.
The connection to the server was lost. Attempting reset: Failed.
```

That will show the last readable row. DBRE and/or DB Support should analyze the best path to go from here.